# Who Maintains the Grammar?
# Bots, Bus Factors, and the Afterlife of Visualization Research Software

Anonymous Submission

**ABSTRACT** The visualization community increasingly ships software with its papers, and some of that software has become infrastructure that the rest of the field builds on. We know a good deal about whether research code is *released*; we know very little about whether it is *maintained*. We mine the full commit histories of 14 repositories in the Protovis → D3 → Vega → Vega-Lite → Mosaic lineage, together with ten comparison toolkits, and report four measurements. Effort is extraordinarily concentrated: 3 people account for half of 26,902 human commits by 666 people. Human effort collapses rather than plateaus: Vega-Lite fell from 1,855 human commits in 2015 to 55 in the last twelve months, a 97% decline. Automation fills the vacuum: 58% of Vega-Lite's recent commits are written by bots, and in three sibling repositories bots write more than four commits in five. And award-winning artifacts go dormant early — Voyager's last human commit predates its Test-of-Time award by four years. We argue that release-time reproducibility checks measure the wrong moment, and sketch a lightweight maintenance disclosure that would make the situation legible.

**INDEX TERMS** Visualization software, research software engineering, repository mining, reproducibility, sustainability, grammars of graphics.

## 1 INTRODUCTION

A visualization paper that ships code is now the norm rather than the exception, and the community has built real machinery to encourage it: the Graphics Replicability Stamp Initiative, IEEE VIS Open Practices, and a growing literature on whether artifacts are available and whether they run [11]. That machinery evaluates a single instant — the moment of publication. It asks whether the code was released and whether a reviewer could execute it.

For most papers that is the right question, because most research prototypes are arguments rather than products. But a handful of systems escape that category. Vega-Lite [14] is not merely a paper with an artifact attached; it is the substrate that a substantial part of the field's subsequent empirical work runs on, from natural-language charting interfaces to notebook toolchains. When a research system becomes infrastructure, availability at publication time stops being the interesting property. The interesting property is whether anyone is still there.

Nobody has measured that. This paper does, for the best-documented lineage the visualization community has: five generations of published systems from one research group, each with its own paper at a visualization venue, spanning seventeen years of continuous version control. We mine the commit histories of 14 repositories in that lineage plus ten widely-used comparison toolkits, and ask who is actually writing the commits.

The answer is uncomfortable, and it is not simply a story of software aging. Our contributions are: (1) the first longitudinal measurement of maintenance effort across a visualization research lineage; (2) four principal findings — extreme effort concentration, collapse rather than plateau of human contribution, bot substitution, and early dormancy of award-winning artifacts — each quantified against comparison toolkits that show the pattern is contingent, not inevitable; and (3) a concrete, cheap proposal for making maintenance status visible at the moment readers actually need it.

## 2 RELATED WORK

*Artifact availability.* Isenberg's study of reproducibility stamps [11] characterizes how visualization papers release code and how that practice has spread. It is the closest neighbour to this work and, we think, exactly half of the picture: it measures release, we measure survival. The community has precedent for turning its own output into data [4, 10], but it has aimed that instrument at papers and figures rather than at code.

*Mining visualization artifacts.* Prior work has mined visualization *artifacts and usage*: corpora of D3 visualizations crawled from the web [8], Stack Overflow questions about D3, and studies of how Observable users decompose code. These treat the toolkit as a fixed backdrop and study what people make with it. We invert this and study the toolkit's own repository.

*Sustainability in software engineering.* Bus factor and contributor concentration are standard instruments in software engineering, as are studies of why open-source projects fail [1, 5, 6]. Concentration around one or two maintainers is unremarkable for open source in general. Our claim is not that the Vega ecosystem is unusual *as open source*; it is that a field which has made this software load-bearing has never checked, and that the numbers are worse than the field's reliance on them would justify.

*The research–practice gap.* The VisGap workshop has convened at EuroVis annually since 2022 to discuss the distance between visualization research and visualization software. That discussion has been conducted almost entirely through position papers and experience reports. We offer it a quantitative baseline.

## 3 CORPUS AND METHOD

We study 14 repositories from a single research lineage — Protovis [2], D3 [3], Vega [15], Vega-Lite [14] and Mosaic [7], plus the prototypes and support packages published alongside them (Lyra, Voyager [19], Draco [13], Falcon, Altair [16], and the editor, embed, tooltip and datasets packages) — and ten comparison toolkits (Matplotlib [9], Bokeh, plotly.py, Chart.js, ECharts [12], deck.gl, ggplot2 [17], Observable Plot, Gephi, and the ten `d3-*` core modules analysed as one family). The lineage was chosen because it is the only case we know of in which every generation has a paper at a visualization venue, which makes research provenance a controlled property rather than a judgement call. It is also a lineage with an unusually explicit intellectual debt: each generation refines a grammar of graphics [18] into executable form, which is part of why its artifacts became infrastructure rather than demonstrations.

Each repository is cloned blobless and its full history walked with `git log --no-merges`. Merge commits are excluded as bookkeeping. Author names are lower-cased and passed through a small, explicit alias table that merges handle/real-name pairs for the same person; this is the only manual judgement in the pipeline, and it is conservative in the direction that *reduces* apparent concentration. Bot commits are identified by author name and reported separately rather than discarded, because their share is itself a result. All windows are measured against a pinned snapshot date of 8 August 2026, so

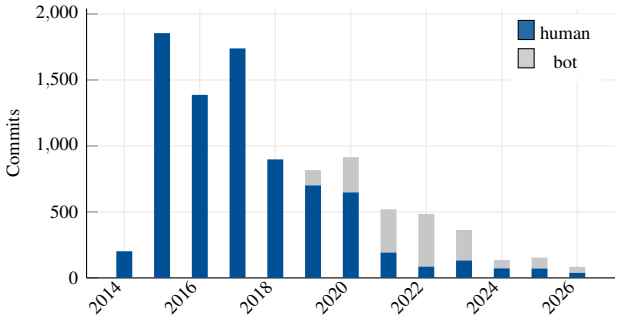



**Figure 1:** Commits to `vega/vega-lite` per year, separated into human and automated authorship. Human contribution peaks in 2015 and declines by 97% to 55 commits in the last twelve months; bots, absent before 2019, now write 58% of the repository's commits. The final bar covers a partial year, to the 8 August 2026 snapshot.

re-running the analysis reproduces the numbers below rather than drifting.

Commit counts are a crude proxy for effort, and we lean on them only for order-of-magnitude claims and for changes over time within a single repository, where the proxy's biases are at least stable. Section 6 revisits this.

## 4 FINDINGS

### 4.1 Half the ecosystem is three people

Across the 14 lineage repositories there are 26,902 human commits from 666 distinct people. Half of that work was done by 3 of them and eighty percent by 8; the top three contributors alone write 66% of all commits, and one person has committed to all 14 repositories. In the last twelve months the whole ecosystem received 483 human commits from 62 people.

The D3 family is more concentrated still: of 7,681 human commits a single author wrote 77%, and the last twelve months brought 2 human commits across all eleven repositories combined. Ten of the `d3-*` core modules share a last human commit in 2023 or 2024.

### 4.2 Contribution collapses, it does not plateau

Figure 1 shows the shape of the decline. If Vega-Lite were simply a finished piece of software we would expect contribution to fall to a low, stable floor. Instead it falls by 97% from its 2015 peak and keeps falling, while the project's own backlog grows: at the snapshot date Vega-Lite had 718 open and 2,872 closed issues, a 20% open ratio, against D3's 5 open issues.[1] D3 can reasonably be described as finished. A project with a fifth of its issues outstanding and two maintainers cannot.

### 4.3 Bots have replaced people

Before 2019 no commit in Vega-Lite was written by a bot. Bots reached 63% of commits in 2021, and over the last twelve months they account for 58%. The pattern is sharper in the support packages that downstream users actually depend on: bots write 95.9% of recent commits to `vega-tooltip`, 87.8% to `vega-embed`, and 81.1% to `vega-datasets`. In three repositories the all-time leading committer is now Dependabot.

This is not nothing — dependency bumps are real maintenance, and a project whose bots keep running is in better shape than one whose bots have stopped. But it is maintenance that cannot triage an issue, review a design proposal, or decide what the grammar should mean.

[1] Issue counts come from the GitHub API on the snapshot date and, unlike everything else reported here, are not reproducible from the git history; they are recorded in `data/issues.json`.

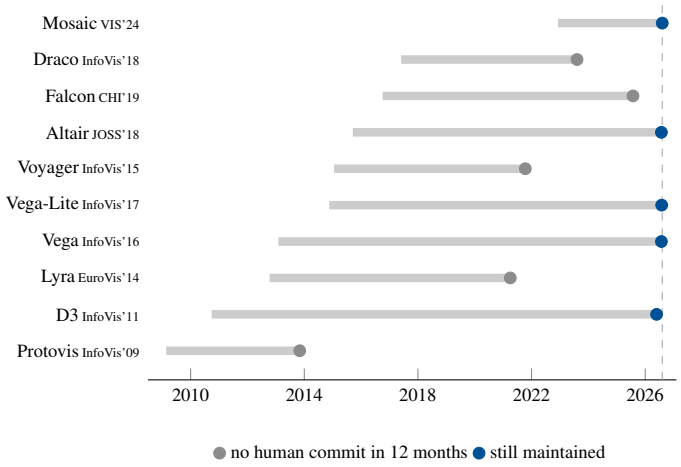



**Figure 2:** Active lifetime of each published system in the lineage, from first commit to last commit by a human. Systems with no human commit in the twelve months before the snapshot are marked in grey. 5 of the 10 are dormant by that measure, among them a best-paper winner (Draco) and a test-of-time winner (Voyager).

### 4.4 The grammar stopped growing

Vega-Lite ships a machine-readable JSON Schema with every release, which makes its expressiveness measurable rather than arguable — a property no other visualization grammar we know of provides. Across 118 releases the schema grew from 170 definitions and 307 KB at v2.0.0 (2017, the release accompanying the paper) to 458 definitions and 1.79 MB — and then stopped. The schema has been byte-identical across the last 6 releases, from v6.2.0 (2025-06-27) to v6.4.3. Whatever the project is now doing, it is not extending the grammar.

### 4.5 Awards arrive after the lights go out

Figure 2 places each published system on a timeline from its first commit to its last human commit. Of the 10, 5 have received no human commit in the last twelve months: Protovis since 2013-11-05, Lyra since 2021-04-02, Draco — a best-paper winner — since 2023-08-07, and Falcon since 2025-07-30.

The sharpest case is Voyager [19], which received the InfoVis Test-of-Time award in 2025. Its last human commit was 2021-10-14: the artifact had been dormant for four years at the moment the community honoured its durability. The award was for the ideas, and the ideas have indeed lasted. But the gap between what we celebrate and what we sustain is precisely the gap this paper is about.

### 4.6 The pattern is contingent, not inevitable

Table 1 makes clear that this is not simply what happens to ten-year-old visualization software. Matplotlib, older still and far larger, received 1,194 human commits in the same twelve months with a bus factor of 7. ECharts and Bokeh, both foundation- or company-governed, remain healthily active. What distinguishes them is not age or language but governance: they have an institutional home that outlives any individual contributor, and research-group software generally does not.

Effort in the lineage has not disappeared so much as migrated. Mosaic, the newest generation, received 180 human commits in the last twelve months — more than Vega-Lite and Vega combined. The lab is still producing excellent research software. It is producing the *next* one.

**Table 1:** Maintenance profile at 8 August 2026. *Bus 50* is the number of authors accounting for half of all human commits; *Top* is the leading author's share; *12mo* counts human commits in the last twelve months and *Bot* the share of commits in that window written by bots.

| Repository | Commits | Authors | Bus 50 | Top | 12mo | Bot |
|---|---|---|---|---|---|---|
| vega-lite | 8,020 | 236 | 2 | 48% | 55 | 58% |
| vega | 5,980 | 165 | 1 | 66% | 80 | 49% |
| d3 | 3,822 | 154 | 1 | 68% | 2 | 0% |
| altair | 3,082 | 204 | 3 | 42% | 117 | 11% |
| mosaic | 1,028 | 34 | 1 | 70% | 180 | 25% |
| matplotlib | 37,691 | 1,976 | 7 | 11% | 1,194 | 3% |
| bokeh | 17,788 | 734 | 3 | 30% | 280 | 0% |
| echarts | 8,355 | 351 | 4 | 17% | 193 | 8% |
| ggplot2 | 5,247 | 384 | 2 | 43% | 92 | 0% |
| plot | 2,125 | 28 | 1 | 80% | 37 | 0% |

## 5 DISCUSSION

*We are measuring the wrong moment.* Reproducibility stamps and artifact-availability checks certify a system at publication, when enthusiasm and funding are at their peak and the student who wrote it has not yet graduated. Every system in our corpus would have passed such a check. The information a reader needs in 2026 — is this maintained, by whom, and should I build on it — is exactly what the badge does not carry.

*A maintenance disclosure.* We propose something deliberately small, because proposals that require sustained volunteer labour are self-defeating given the finding above. Every number in this paper was computed from public git history by a script that runs in minutes. A venue could attach to each artifact a generated, dated line reporting human commits in the last twelve months, distinct recent authors, bot share, and date of last human commit — and, critically, recompute it on the digital library page rather than freezing it at publication. This is not a quality judgement and should not be read as one. Dormant is not the same as bad: Protovis is dormant because D3 replaced it, which is a success. The point is to make the distinction between *finished*, *superseded* and *abandoned* visible instead of leaving each reader to guess.

*An honest tension.* Researchers are rewarded for the next paper, not for the previous system's issue tracker, and we do not think individual authors are at fault here — the migration of effort to Mosaic is exactly what the incentive structure asks for. If the community wants its infrastructure maintained, the realistic routes are institutional homes for the few systems that become load-bearing, or explicit acknowledgement that research software has a half-life and planning for succession rather than pretending otherwise.

## 6 THREATS TO VALIDITY

Commit counts measure activity, not value; a single well-judged commit can be worth a hundred. We use them for large ratios and within-repository trends rather than cross-project quality comparisons, but the proxy remains crude, and work that never reaches a commit — issue triage, design review, user support — is invisible to it. Our alias table merges identities we could confirm and will have missed others, which biases concentration *downward*, making our central finding conservative. Bot classification by author name will misclassify humans committing through automation. The twelve-month window is a choice; a longer window softens the picture, a shorter one sharpens it. Finally, one lineage is not the field — it is the best-documented case, not a random sample, and the comparison toolkits are a convenience sample of widely used projects rather than a systematic one. We report a pattern within a well-characterized corpus and do not claim it generalizes to visualization research software as a whole.

## 7 CONCLUSION

We measured who maintains the visualization community's most-used research software and found 3 people behind half of it, human contribution down 97% in its flagship project, bots writing the majority of recent commits, a grammar that has stopped growing, and a test-of-time award given to an artifact that had been dormant for four years. Comparison with foundation-governed toolkits of the same age shows none of this is a law of software aging.

Our field has taught itself to release code. The measurements here suggest the harder and now more urgent question is what happens in the decade afterwards.

## REFERENCES


[1] G. Avelino, L. Passos, A. Hora, and M. T. Valente. A novel approach for estimating truck factors. In *Proceedings of the IEEE 24th International Conference on Program Comprehension (ICPC)*, pages 1–10, 2016.

[2] M. Bostock and J. Heer. Protovis: A graphical toolkit for visualization. *IEEE Transactions on Visualization and Computer Graphics*, 15(6):1121–1128, 2009.

[3] M. Bostock, V. Ogievetsky, and J. Heer. $D^3$ data-driven documents. *IEEE Transactions on Visualization and Computer Graphics*, 17(12):2301–2309, 2011.

[4] J. Chen, M. Ling, R. Li, P. Isenberg, T. Isenberg, M. Sedlmair, T. Möller, R. S. Laramee, H.-W. Shen, K. Whlisch, and G. Andrienko. VIS30K: A collection of figures and tables from IEEE visualization conference publications. *IEEE Transactions on Visualization and Computer Graphics*, 27(9):3826–3833, 2021.

[5] J. Coelho and M. T. Valente. Why modern open source projects fail. In *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, pages 186–196, 2017.

[6] A. E. Hassan. The road ahead for mining software repositories. In *Frontiers of Software Maintenance*, pages 48–57, 2008.

[7] J. Heer and D. Moritz. Mosaic: An architecture for scalable and interoperable data views. *IEEE Transactions on Visualization and Computer Graphics*, 30(1):436–446, 2024.

[8] E. Hoque and M. Agrawala. Searching the visual style and structure of D3 visualizations. *IEEE Transactions on Visualization and Computer Graphics*, 26(1):1236–1245, 2020.

[9] J. D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

[10] P. Isenberg, F. Heimerl, S. Koch, T. Isenberg, P. Xu, C. D. Stolper, M. Sedlmair, J. Chen, T. Möller, and J. Stasko. vispubdata.org: A metadata collection about IEEE visualization (VIS) publications. *IEEE Transactions on Visualization and Computer Graphics*, 23(9):2199–2206, 2017.

[11] T. Isenberg. The state of reproducibility stamps for visualization research papers. *arXiv preprint arXiv:2408.03889*, 2024.

[12] D. Li, H. Mei, Y. Shen, S. Su, W. Zhang, J. Wang, M. Zu, and W. Chen. ECharts: A declarative framework for rapid construction of web-based visualization. *Visual Informatics*, 2(2):136–146, 2018.

[13] D. Moritz, C. Wang, G. L. Nelson, H. Lin, A. M. Smith, B. Howe, and J. Heer. Formalizing visualization design knowledge as constraints: Actionable and extensible models in Draco. *IEEE Transactions on Visualization and Computer Graphics*, 25(1):438–448, 2019.

[14] A. Satyanarayan, D. Moritz, K. Wongsuphasawat, and J. Heer. Vega-lite: A grammar of interactive graphics. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):341–350, 2017.

[15] A. Satyanarayan, R. Russell, J. Hoffswell, and J. Heer. Reactive vega: A streaming dataflow architecture for declarative interactive visualization. *IEEE Transactions on Visualization and Computer Graphics*, 22(1):659–668, 2016.

[16] J. VanderPlas, B. Granger, J. Heer, D. Moritz, K. Wongsuphasawat, A. Satyanarayan, E. Lees, I. Timofeev, B. Welsh, and S. Sievert. Altair: Interactive statistical visualizations for Python. *Journal of Open Source Software*, 3(32):1057, 2018.

[17] H. Wickham. A layered grammar of graphics. *Journal of Computational and Graphical Statistics*, 19(1):3–28, 2010.

[18] L. Wilkinson. *The Grammar of Graphics*. Springer, 2nd edition, 2005.

[19] K. Wongsuphasawat, D. Moritz, A. Anand, J. Mackinlay, B. Howe, and J. Heer. Voyager: Exploratory analysis via faceted browsing of visualization recommendations. *IEEE Transactions on Visualization and Computer Graphics*, 22(1):649–658, 2016.